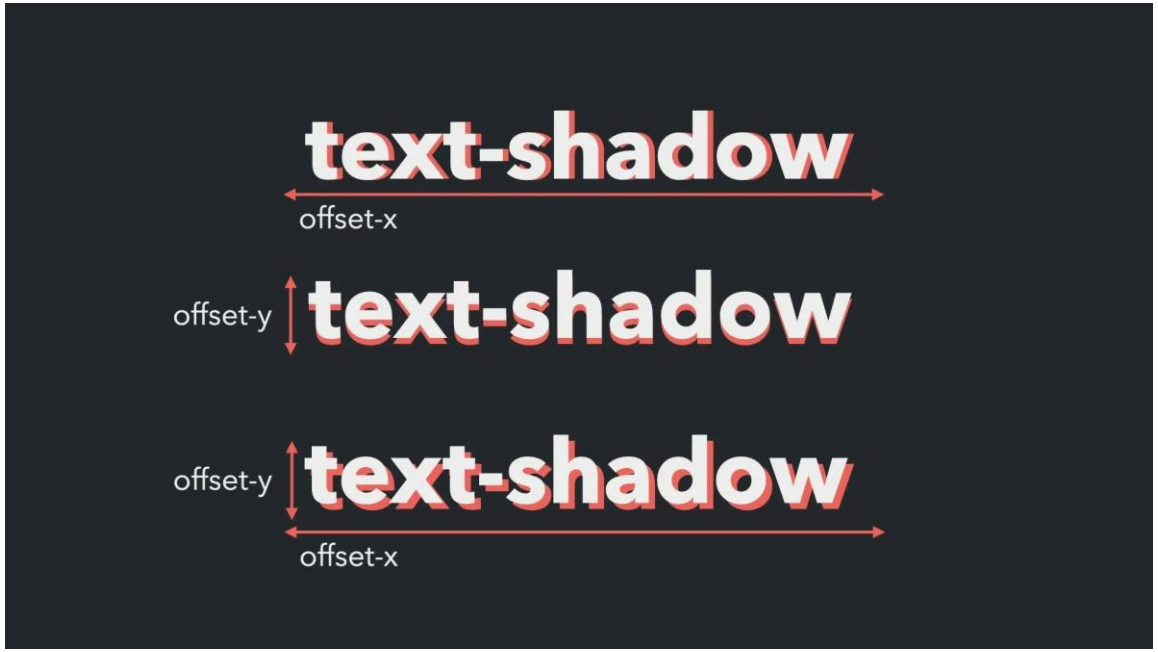
Эффекты и тени, адаптивная вёрстка

Докладчики: Корнев Андрей, Голубков Кирилл, Шипелин Алексей

CSS-свойство text-shadow

- CSS-свойство **text-shadow** добавляет тени к тексту. Свойство задаётся разделённым запятыми списком теней, которые будут применены к тексту и к любым его свойствам decorations. Любая тень описывается комбинацией смещений по осям X и Y относительно элемента, радиусом размытия и цветом.Это свойство определено как разделённый запятыми список теней.



CSS-свойство text-shadow

- Каждая тень определена как два или три значения <длина>, за которыми следует необязательное значение <цвет>.
- Первые два значения определяют параметры <смещение-x> и <смещение-y>.
- Значение <цвет> определяет цвет тени.

```
.text2 {
    text-shadow: 5px 10px;
}
```

Far out in the uncharted backwaters of the unfashionable end of the western spiral arm of the Galaxy...

CSS-свойство text-shadow

- Третье значение задаёт <радиус-размытия>.
- Если определены больше чем одна тень, тени накладываются друг на друга в порядке обратном порядку их определения, что значит, первая тень окажется поверх последующих.

```
.text3 {
    text-shadow:
        1px 1px 2px red,
        0 0 16px blue,
        0 0 3px blue;
}
```

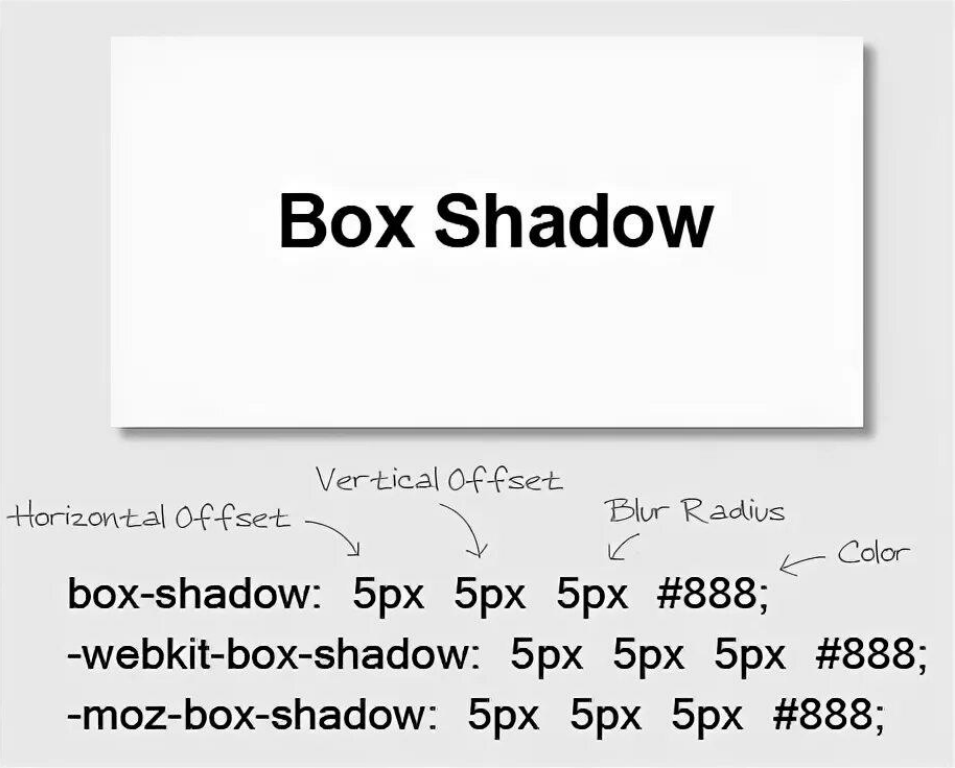
Far out in the uncharted backwaters of the unfashionable end of the western spiral arm of the Galaxy...

```
.text3 {
    text-shadow:
        1px 1px 2px red,
        0 0 16px blue,
        0 0 3px blue;
}
```

```
<h1 class="text3">Far out in the uncharted
backwaters of the unfashionable end of the
western spiral arm of the
        Galaxy...</h1>
```

CSS-свойство box-shadow

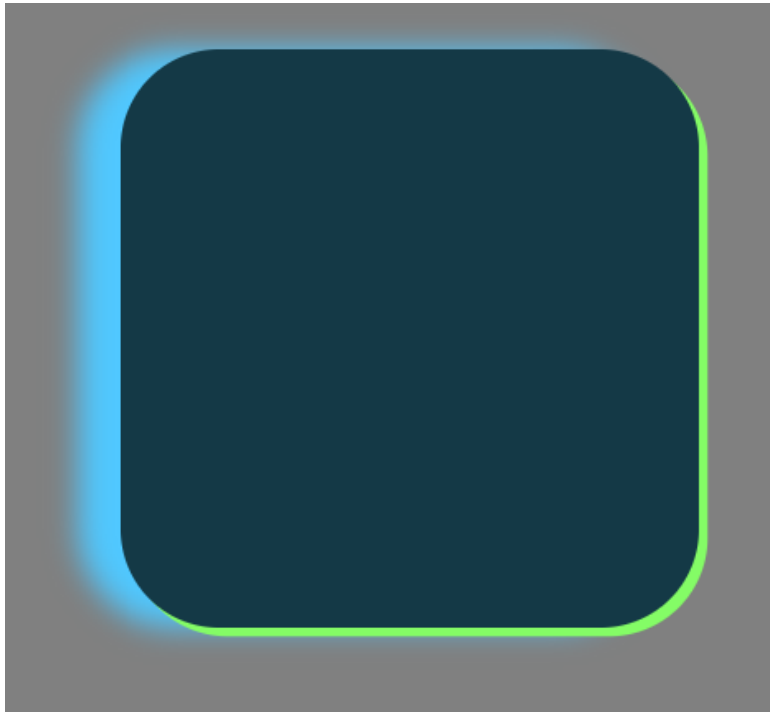
- CSS-свойство **box-shadow** добавляет тень к элементу. Через запятую можно задать несколько теней. Тень описывается смещениями по оси X и Y относительно элемента, радиусом размытия и распространения, а также цветом.



CSS-свойство box-shadow

- Свойство box-shadow включает отбрасывание тени от границ практически любого элемента. Если для элемента с тенью задан border-radius, то отбрасываемая тень также будет с закруглёнными углами.

```
.box5 {
    width: 200px;
    height: 200px;
    background-color: rgb(20, 57, 70);
    border-radius: 34px;
    /* Любое количество теней через запятую */
    box-shadow:
        3px 3px rgb(132, 252, 102),
        -15px 0 10px rgb(80, 200, 255);
}
```



CSS-свойство box-shadow

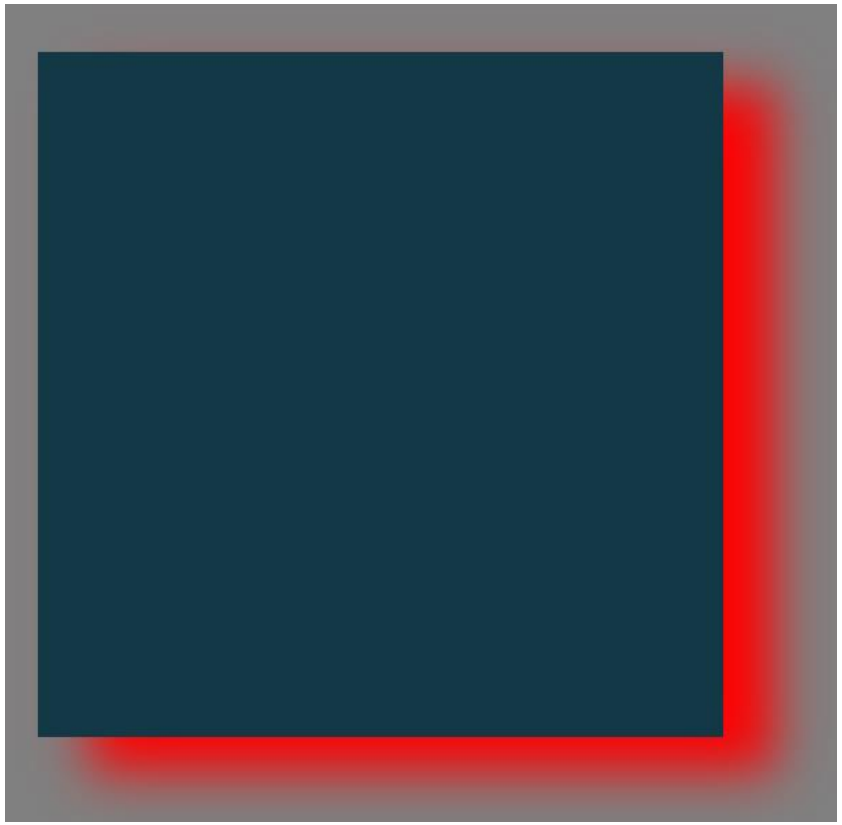
- Каждая тень определяется следующим образом:
- Два, три или четыре значения длины (<length>).
- Если указаны только два значения, они будут означать смещение по X (<offset-x>) и смещение по Y (<offset-y>). Необязательное значение цвета (<color>).

```
.box1 {
    width: 200px;
    height: 200px;
    background-color: rgb(20, 57, 70);
    /* offset-x | offset-y  color */
    box-shadow: 20px -16px teal;
}
```

CSS-свойство box-shadow

- Третье значение устанавливает радиус размытия (<blur-radius>).

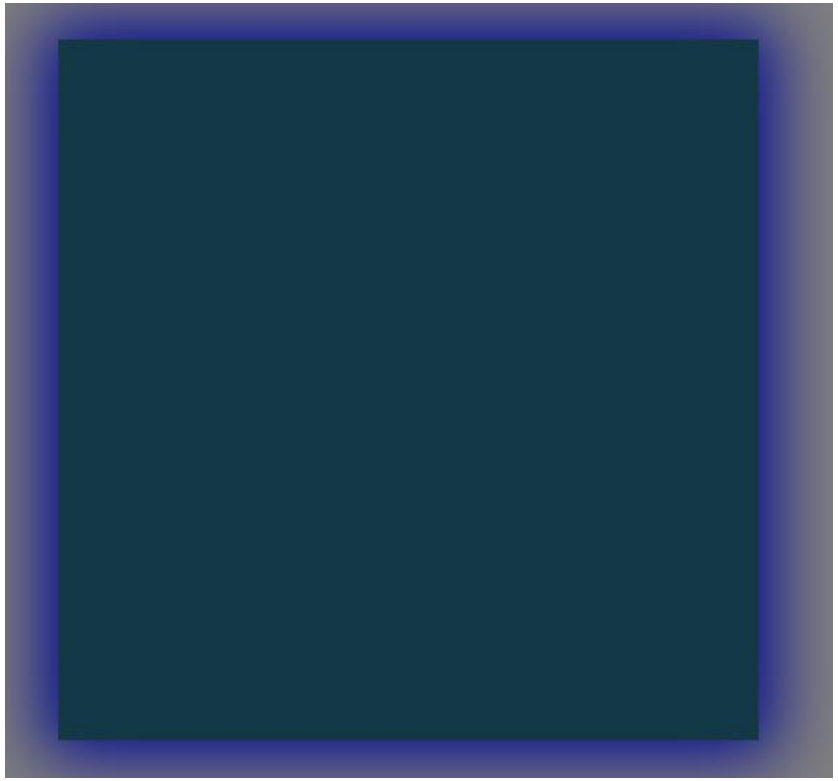
```
.box2 {
    width: 200px;
    height: 200px;
    background-color: rgb(20, 57, 70);
    /* offset-x | offset-y | blur-radius | color */
    box-shadow: 15px 10px 15px red;
}
```



CSS-свойство box-shadow

- Четвёртое значение задаёт радиус распространения (<spread-radius>).

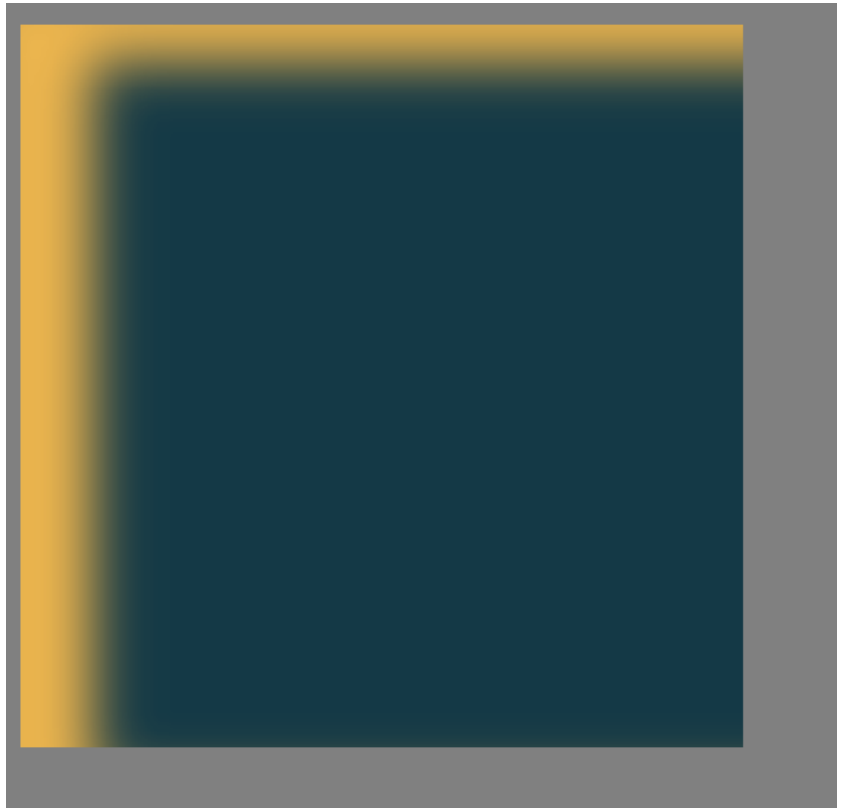
```
.box3 {
    width: 200px;
    height: 200px;
    background-color: rgb(20, 57, 70);
    /* offset-x | offset-y | blur-radius | spread-radius | color */
    box-shadow: 0px 0px 20px 5px rgb(2, 9, 143);
}
```



CSS-свойство box-shadow

- Необязательное ключевое слово inset.

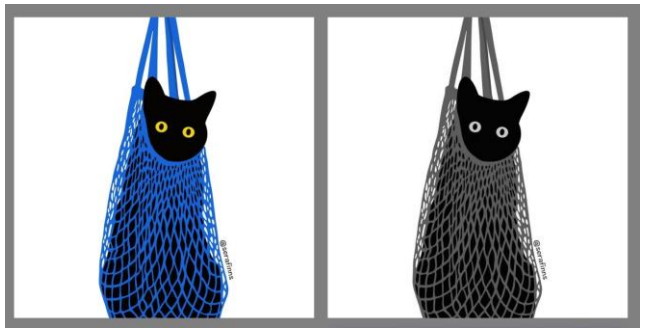
```
.box4 {
    width: 200px;
    height: 200px;
    background-color: rgb(20, 57, 70);
    /* inset | offset-x | offset-y | color */
    box-shadow: inset 20px 10px 15px rgb(238, 182, 78);
}
```



CSS-свойство Filter

- Filter позволяет нам применять к элементу такие графические элементы, как размытие и смещения цвета. Фильтры обычно используются для регулировки рендеринга изображений, фонов и рамок

```
.photo_grayscale {
    filter: grayscale(100%);

}
```



grayscale() — делает элемент чёрно-белым

```
.photo_blur {
    filter: blur(5px);
}
```

blur() — размывает элемент

CSS-свойство Filter

```
.photo_brightness {
    filter: brightness(0.4);
}
```

brightness() — изменяет яркость элемента

```
.photo_contrast {
    filter: contrast(200%);
}
```

contrast() — изменяет контраст элемента

CSS-свойство Filter

```
.photo_drop-shadow {
    filter: drop-shadow(16px 16px 20px rgb(131, 131, 240));
}
```

drop-shadow() — добавляет тень по форме элемента

```
.photo_invert {
    filter: invert(100%);
}
```

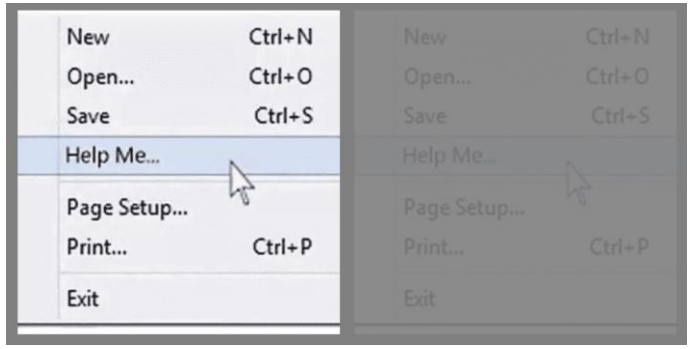
invert() — инвертирует цвета элемента

CSS-свойство Filter

```
.photo_hue-rotate {
    filter: hue-rotate(90deg);
}
```

hue-rotate() — изменяет оттенок (сдвигает цвета по цветовому кругу)

```
.photo_opacity {
    filter: opacity(10%);
}
```

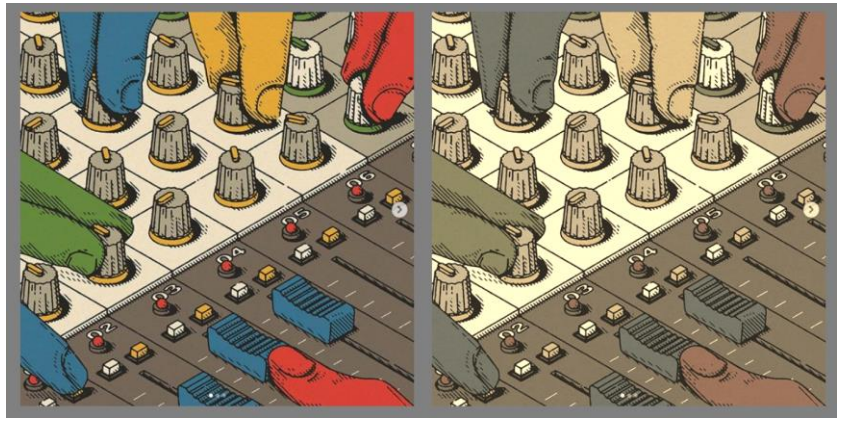


opacity() — изменяет прозрачность элемента

CSS-свойство Filter

```
.photo_saturate {
    filter: saturate(300%);
}
```

```
.photo_sepia {
    filter: sepia(80%);
}
```



saturate() — изменяет насыщенность цветов элемента

sepia() — придаёт элементу эффект сепии

CSS-свойство Filter

```
.photo_multiple {
    filter: contrast(155%) brightness(130%);
}
```

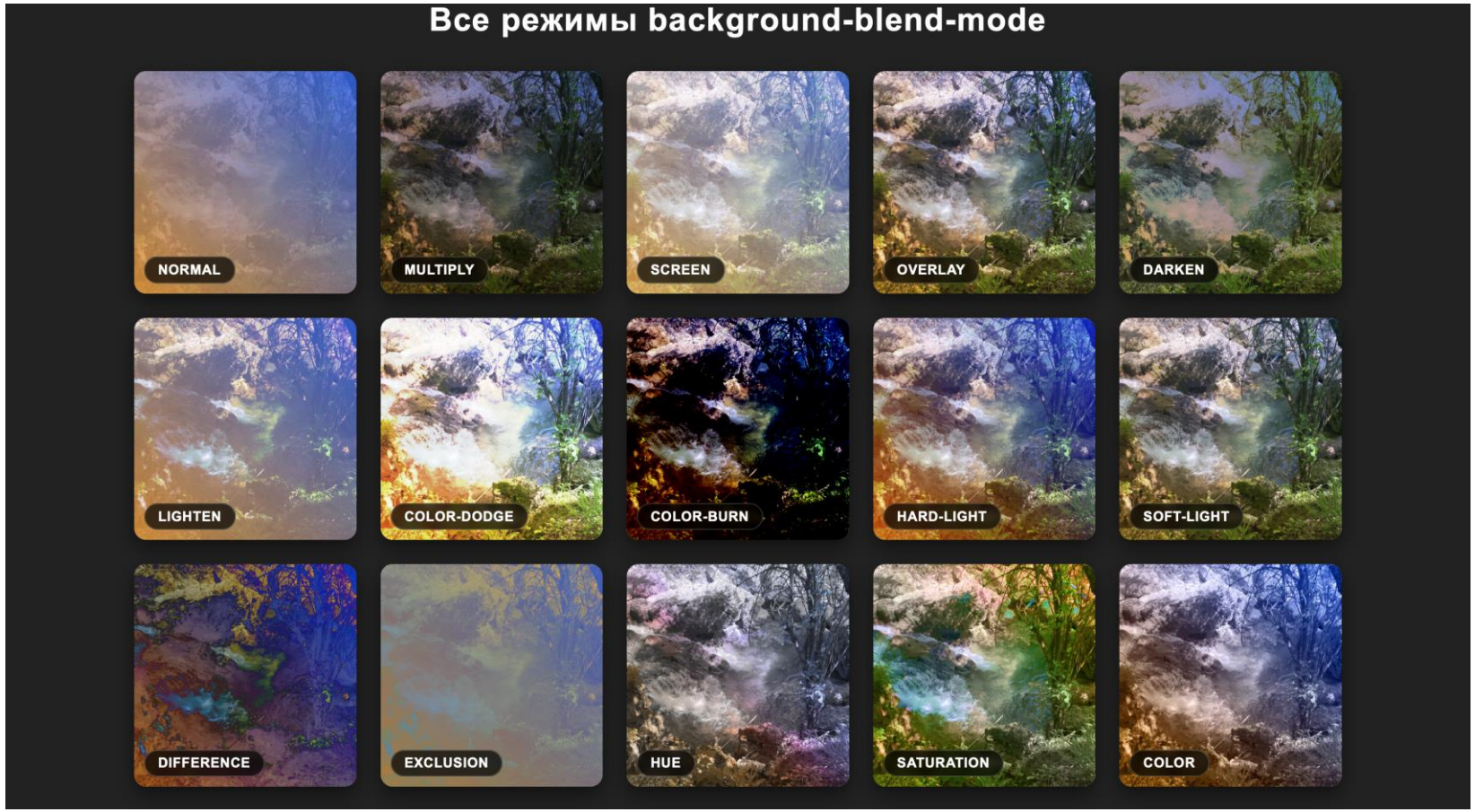

```
.photo_multiple {
    filter: contrast(155%) brightness(130%);
}
```

```
<div class="item">
    <img src="../resources/images/photo2.jpg" class="photo_blur" alt="">
</div>
```

комбинирование фильтров — позволяет применять несколько эффектов одновременно

CSS-свойство background-blend-mode

Свойство background-blend-mode задаёт способ смешивания **нескольких фоновых слоёв** элемента (цвета и изображений).
Работает **только внутри background одного элемента**.



CSS-свойство background-blend-mode

Возможные режимы наложения:

1. **Затемняющие режимы**

-darken(Берётся минимальное значение из каждого канала (R, G, B) обоих слоёв. Итоговый цвет темнее каждого из исходных)

```
.blend-darken {
  background-blend-mode: darken;
}
```

-multiply(Значения каналов перемножаются и делятся на 255. Эффект похож на рисование фломастерами – всегда получается более тёмный цвет (кроме белого, который «исчезает»).

```
.blend-multiply {
  background-blend-mode: multiply;
}
```

-color-burn(Увеличивает контраст, делая нижний слой темнее. Чем темнее верхний слой, тем сильнее эффект. Белый цвет не изменяет картинку)

```
.blend-color-burn {
  background-blend-mode: color-burn;
}
```

CSS-свойство background-blend-mode

2. **Осветляющие режимы**

-lighten(Берётся максимальное значение из каждого канала. Итог светлее каждого исходного)

```
.blend-lighten {
  background-blend-mode: lighten;
}
```

-screen(Противоположность multiply: перемножаются инвертированные значения. Всегда осветляет, чёрный цвет не влияет)

```
.blend-screen {
  background-blend-mode: screen;
}
```

-color-dodge(Осветляет нижний слой, уменьшая контраст. Чем светлее верхний слой, тем сильнее эффект. Чёрный не изменяет фон.)

```
.blend-color-dodge {
  background-blend-mode: color-dodge;
}
```

CSS-свойство background-blend-mode

3. **Контрастные режимы**

-soft-light(Мягкий вариант hard-light – аналогичен рассеянному свету. Немного затемняет или осветляет в зависимости от тона верхнего слоя)

```
.blend-soft-light {
  background-blend-mode: soft-light;
}
```

-overlay(Комбинация multiply и screen: тёмные участки становятся темнее (multiply), светлые – светлее (screen). Сохраняет контраст исходного нижнего слоя)

```
.blend-overlay {
  background-blend-mode: overlay;
}
```

-hard-light(Противоположность overlay: применяет multiply к тёмным участкам верхнего слоя и screen к светлым. Эффект похож на яркий прожектор.)

```
.blend-hard-light {
  background-blend-mode: hard-light;
}
```

CSS-свойство background-blend-mode

4. **Инверсионные и сравнительные режимы**

-difference(Вычитает меньшее значение канала из большего (абсолютная разница). Чёрный не даёт изменений, белый – инвертирует цвета)

```
.blend-difference {
  background-blend-mode: difference;
}
```

-exclusion(Похож на difference, но с меньшей контрастностью. Результат мягче, серый даёт нейтральный эффект.)

```
.blend-exclusion {
  background-blend-mode: exclusion;
}
```

CSS-свойство background-blend-mode

5. **Компонентные режимы**

-hue(Сохраняет яркость и насыщенность нижнего слоя, а цветовой тон (hue) берёт от верхнего)

```
.blend-hue {
  background-blend-mode: hue;
}
```

-saturation(Сохраняет яркость и тон нижнего слоя, а насыщенность – от верхнего)

```
.blend-saturation {
  background-blend-mode: saturation;
}
```

-color(Сохраняет яркость нижнего слоя, а тон и насыщенность – от верхнего. Полезно для тонирования чёрно-белых изображений)

```
.blend-color {
  background-blend-mode: color;
}
```

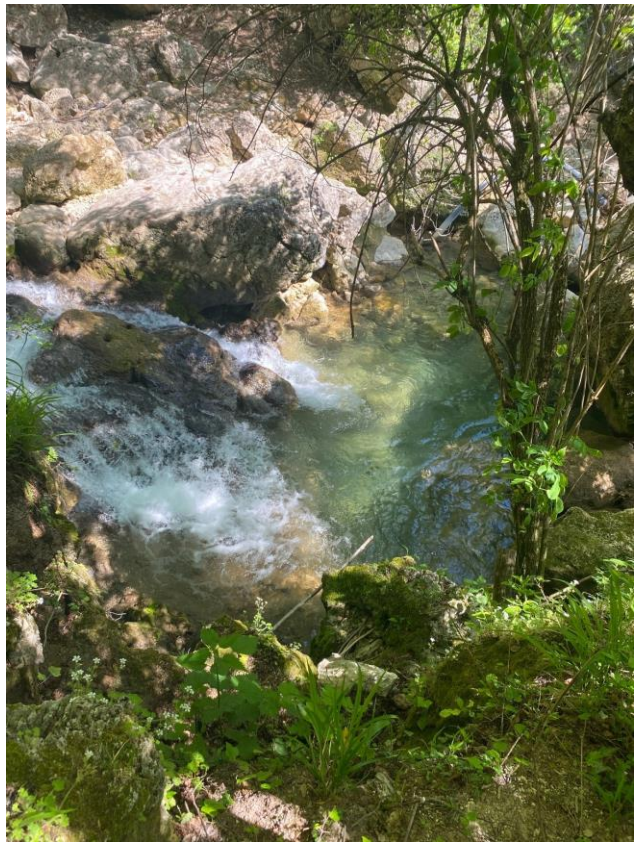
-luminosity(Сохраняет тон и насыщенность нижнего слоя, а яркость – от верхнего. Противоположность color)

```
.blend-luminosity {
  background-blend-mode: luminosity;
}
```

CSS-свойство background-blend-mode

В данном примере использовалось готовое изображение и цвет – это полупрозрачная цветная заливка (оранжево-синяя).

```
.blend-card {
  position: relative;
  width: 100%;
  aspect-ratio: 1 / 1;          /* квадрат */
  background:
    linear-gradient(45deg, rgba(255, 150, 0, 0.8), rgba(0, 100, 255, 0.8)),
    url("../resources/images/hero-desktop.jpg") center/cover no-repeat;
  border-radius: 12px;
  box-shadow: 0 8px 20px rgba(0,0,0,0.5);
  transition: transform 0.2s ease;
}
```



CSS-свойство mix-blend-mode

mix-blend-mode задаёт, **как элемент смешивается с фоном родителя**.
В отличие от background-blend-mode, он работает **между элементами**, а не внутри одного background.

CSS-свойство mix-blend-mode

Mix-blend-mode использует такие же значения, как и background-blend-mode, но в отличии от второго, Смешивает **весь элемент** (включая его контент) с тем, что **под ним**(фон родителя или соседние элементы). Текст, картинка или блок накладываются на другой фон
В данном примере:

1.Есть общий контейнер .mix-container с тёмно-синим фоном.

2.Внутри — цветные круги с одинаковым оранжевым цветом (background: #ffaa00).

3.Каждый круг получает свой mix-blend-mode.

4.Круги накладываются друг на друга (из-за flex-wrap они частично перекрываются) и на фон контейнера.

```
.mix-container {
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
  gap: 1rem;
  max-width: 800px;
  margin: 0 auto;
  background: rgba(255,255,255,0.1);
  padding: 2rem;
  border-radius: 20px;
}
```

```
.mix-circle {
  width: 80px;
  height: 80px;
  border-radius: 50%;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 0.7rem;
  font-weight: bold;
  text-transform: uppercase;
  color: black;
  background: #ffaa00;  /* базовый цвет круга */
  box-shadow: 0 4px 10px rgba(0,0,0,0.3);
```

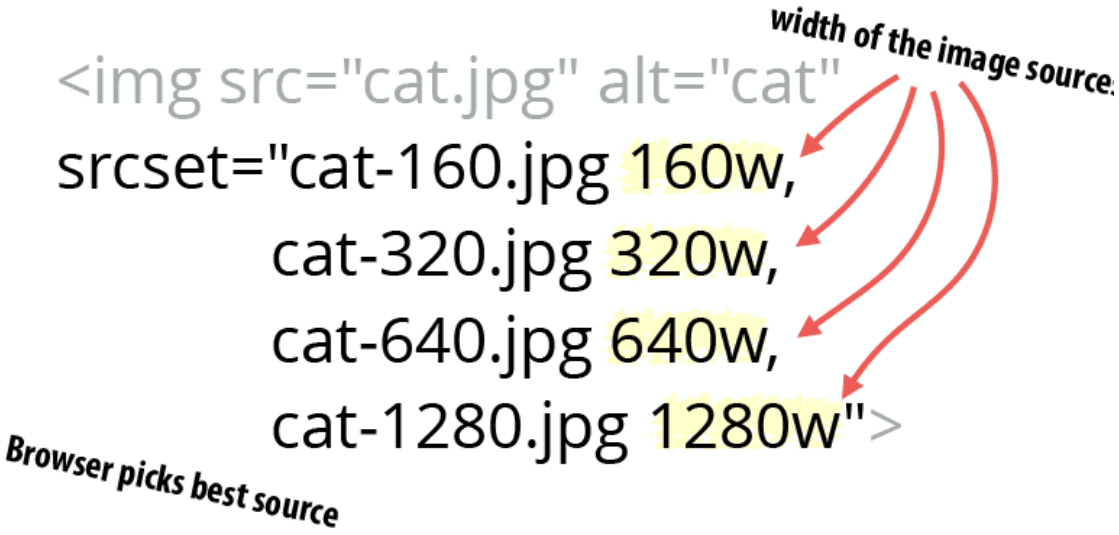
Мета-Тег viewport

- viewport управляет **масштабом и шириной страницы на мобильных устройствах**.
- **Зачем нужен:** Без него мобильные браузеры показывают страницу как уменьшенную десктопную версию, что неудобно для чтения и навигации.
- **Как работает:** Параметры width=device-width и initial-scale=1.0 заставляют страницу подстраиваться под ширину экрана и устанавливают начальный масштаб 100%.
- **Дополнительные настройки:** Можно ограничивать масштабирование (maximum-scale, minimum-scale) или запрещать его (user-scalable=no), но важно помнить о доступности.

```
  <!-- ✅ viewport -->
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Hero Blend Demo</title>
  <link rel="stylesheet" href="css/2_hero.css">
</head>
<body>
```

Scrset (адаптивные изображения)

- srcset позволяет браузеру выбрать **наиболее подходящее изображение** под устройство.
- Экономит трафик
  Ускоряет загрузку
  Улучшает адаптив



Picture

- <picture> позволяет:
- менять изображения по условиям
- использовать разные форматы
- делать art-direction
- Более мощный, чем просто img.

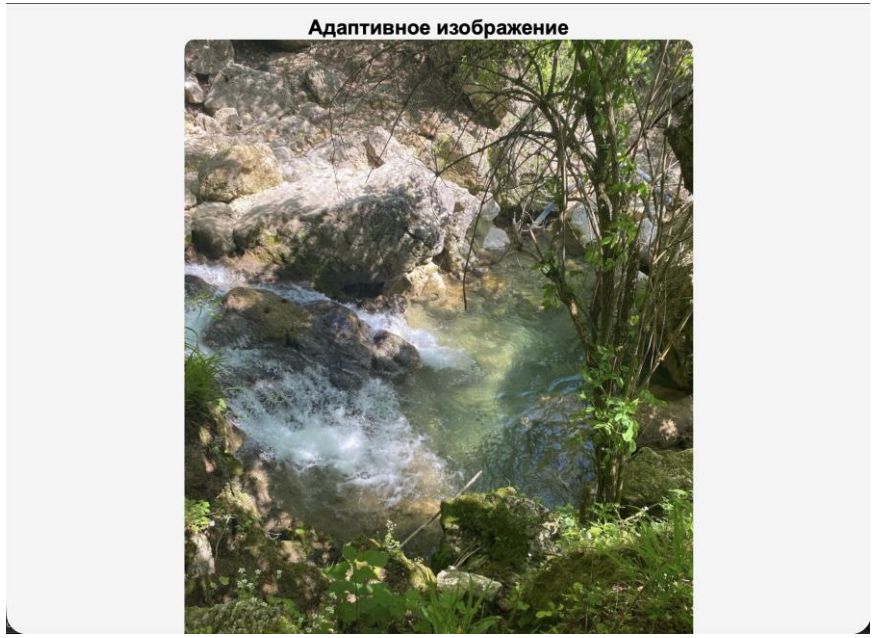
```html
<!-- ✅ picture + srcset -->
<picture>
  <source
    media="(max-width: 600px)"
    srcset="resource/images/hero-mobile.jpg">

  <source
    media="(max-width: 1024px)"
    srcset="resource/images/hero-tablet.jpg">

  <img
    src="resource/images/hero-desktop.jpg"
    alt="Пример адаптивного изображения"
    loading="lazy"
    class="demo-image__img">
</picture>
</section>
```

HTML-атрибут loading="lazy"

- Атрибут loading="lazy" включает **ленивую загрузку изображений**.
- Картинка грузится только когда приближается к экрану

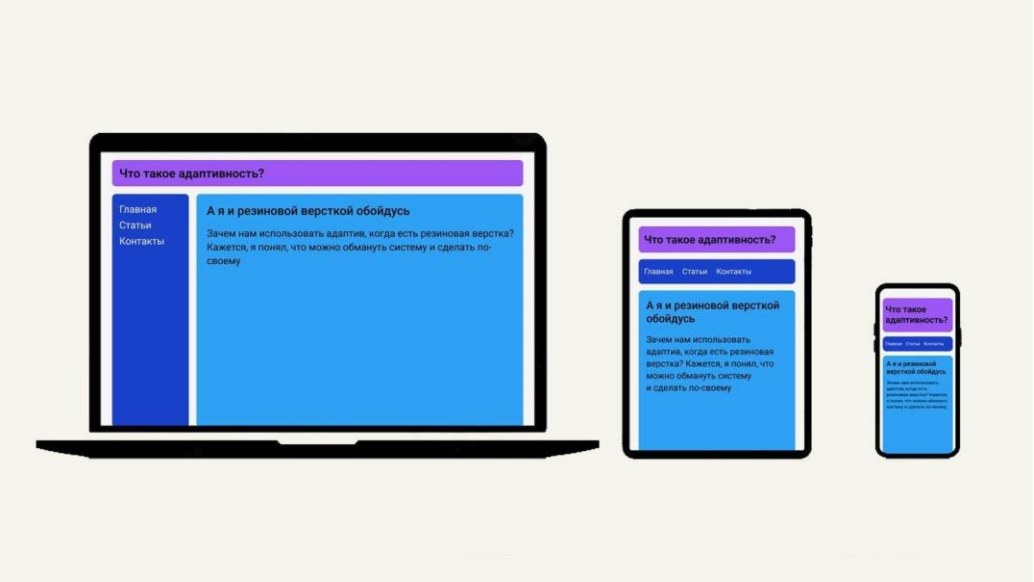
- улучшает:
- скорость страницы
- Core Web Vitals
- UX

```
<img
  src="resource/images/hero-desktop.jpg"
  alt="Пример адаптивного изображения"
  loading="lazy"
  class="demo-image__img">
```



Адаптивная верстка

- Адаптивная верстка — это подход к разработке интерфейсов, при котором сайт корректно отображается на разных устройствах: телефонах, планшетах, ноутбуках и больших мониторах.
- Главная идея — не создавать отдельные версии сайта, а сделать один гибкий интерфейс, который подстраивается под размер экрана пользователя.
- Почему это важно:
- - более 60% трафика — мобильные устройства;
- - улучшение пользовательского опыта;
- - повышение доступности;
- - положительное влияние на SEO.



Media Queries

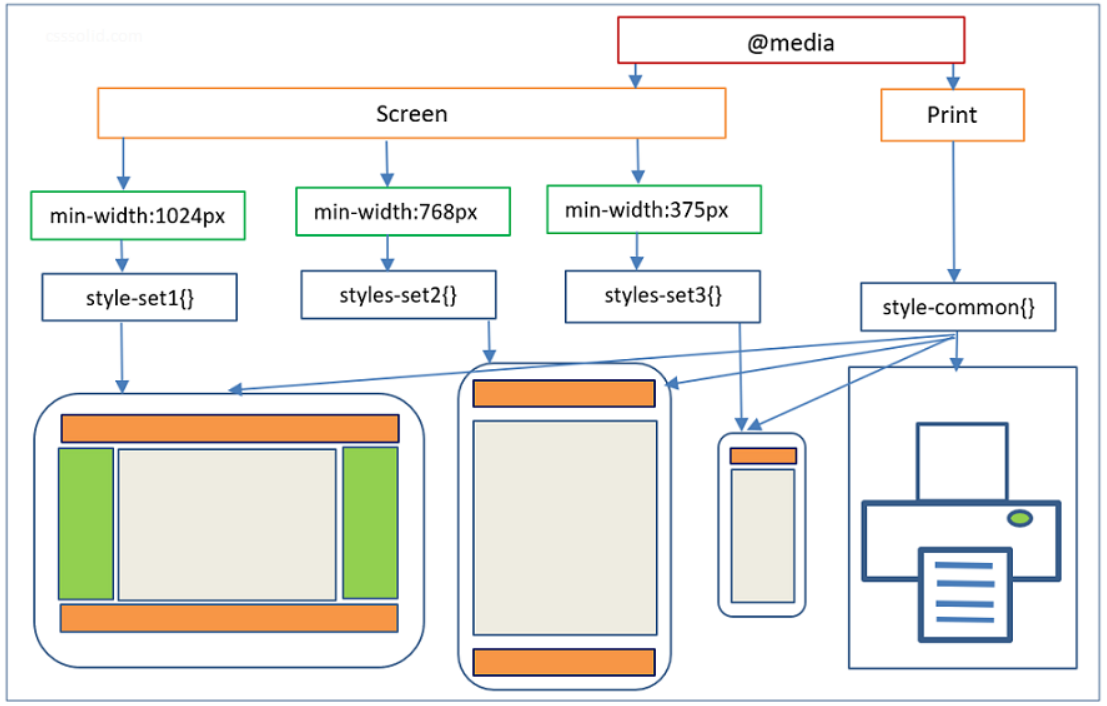
- **Что это такое?**
- Media Queries — это механизм CSS, который позволяет применять стили в зависимости от характеристик устройства или окна браузера.
- Проще говоря, мы задаём условие, и если оно выполняется — применяются определённые стили.

```
@media (max-width: 768px) {
  body {
    background: lightblue;
  }
}
```

- Это означает: если ширина экрана меньше или равна 768px — фон станет голубым.

Media Queries

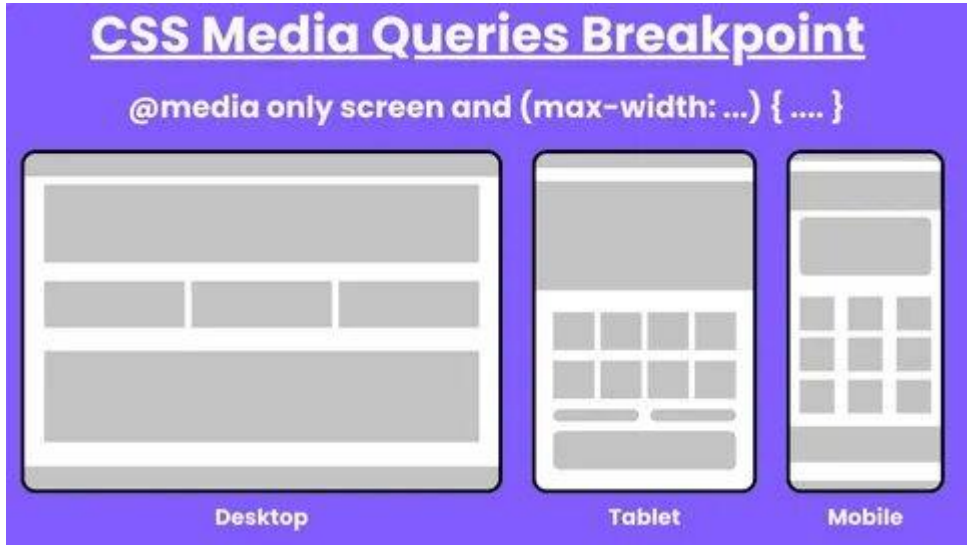
- **Какие условия можно использовать:**
- width / height
- min-width / max-width
- orientation (portrait / landscape)
- resolution
- **Где применяются:**
- изменение расположения блоков
- изменение размера текста
- скрытие или показ элементов
- адаптация меню



- Media Queries — это основной инструмент адаптивной верстки.

Breakpoints (точки перелома)

- **Что это?**
- Breakpoint — это ширина экрана, при которой интерфейс меняет структуру.
- Например:
- - при 1200px — 3 колонки
- - при 768px — 2 колонки
- - при 480px — 1 колонка



- Важно понимать, что точки перелома выбираются не **“по устройствам”**, а по тому моменту, когда дизайн начинает выглядеть плохо.

Подходы к адаптивной верстке

- **Mobile-first**

Сначала пишутся стили для мобильных устройств, затем через min-width добавляются стили для больших экранов.

```
.container {
  width: 100%;
}

@media (min-width: 768px) {
  .container {
    width: 750px;
  }
}
```

Преимущества: - быстрее загружается на мобильных - проще масштабировать - современный стандарт - код становится чище

- **Desktop-first**

Сначала создаётся версия для больших экранов, затем через max-width происходит упрощение интерфейса.

```
.container {
  width: 1200px;
}

@media (max-width: 768px) {
  .container {
    width: 100%;
  }
}
```

Минус — сложнее поддерживать, часто получается больше переопределений.

Гибкие единицы измерения

Адаптивность невозможна без относительных единиц измерения.

%:
Процент вычисляется относительно родительского элемента.
Используется для ширины блоков и сеток.

vw и vh:
1vw = 1% ширины окна
1vh = 1% высоты окна

vmin и vmax:
vmin — меньшее значение между vw и vh
vmax — большее значение
Используются реже, но полезны для адаптивной типографики.

rem:
Зависит только от размера шрифта в теге html.
Если html = 16px: - 1rem = 16px - 2rem = 32px
Преимущество — предсказуемость и удобство масштабирования.

em:
Зависит от font-size родителя.
Если родитель 16px, то 2em = 32px.
Особенность — значения могут накапливаться (вложенность влияет на размер).

Гибкие единицы измерения

Сравнение единиц:

em vs rem:

- em зависит от родителя. rem зависит от html.
- В крупных проектах чаще используют rem, чтобы избежать накопления размеров.

px vs rem:

- px — фиксированная единица. rem — масштабируемая.
- Для доступности и адаптивности лучше использовать rem.

vh vs %:

- vh зависит от окна браузера. % зависит от родителя.
- Для полноэкранных блоков удобнее vh.

Как указать стили для диапазона ширин и ориентации?

Диапазон ширин

Иногда нужно применить стили не для одного значения, а для определенного диапазона.

```
@media (min-width: 768px) and (max-width: 1024px) {
  body {
    background: lightblue;
  }
}
```

Объяснение:

min-width: 768px — от 768px; max-width: 1024px — до 1024px

Здесь стили применяются только в заданном диапазоне.

Как указать стили для диапазона ширин и ориентации?

Ориентация устройства

В CSS мы можем определить, как пользователь держит устройство:

- portrait — вертикально
- landscape — горизонтально

```
@media (orientation: portrait) {
  body {
    background: lightgreen;
  }
}
```

```
@media (orientation: landscape) {
  body {
    background: lightcoral;
  }
}
```

Как указать стили для диапазона ширин и ориентации?

Ориентация устройства
Комбинирование условий

Мы можем объединять диапазон ширин и ориентацию:

```
@media (max-width: 768px) and (orientation: portrait) {
  .container {
    flex-direction: column;
  }
}
```

Это означает, что экран будет ≤ 768px и устройство будет вертикальное.

Как сделать версию для печати?

Media print

CSS позволяет задать отдельные стили для печати страницы.

```
@media print {
  body {
    background: white;
    color: black;
  }
}
```

Обычно при печати убирают всё лишнее и упрощают страницу:

```
@media print {

  .container {
    max-width: 100%;
  }

  .content {
    box-shadow: none;
    border: none;
  }

}
```

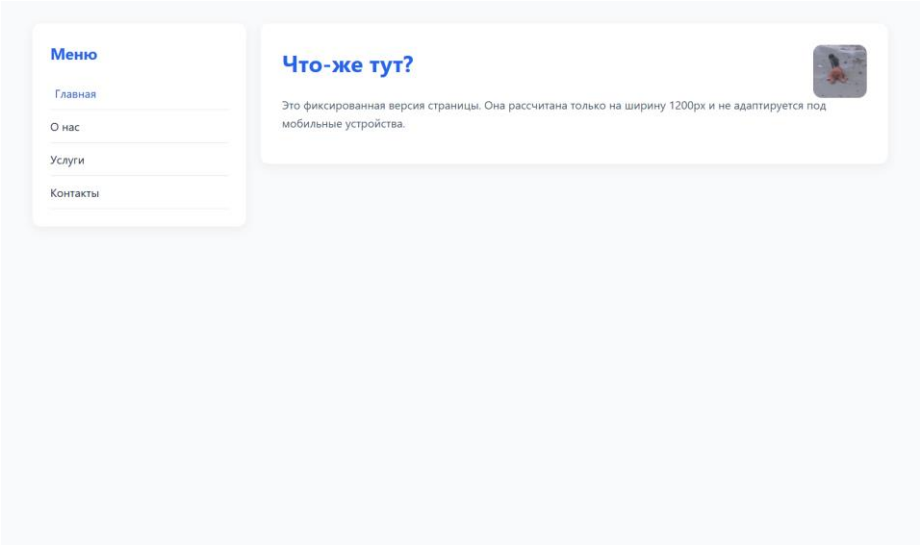
```
@media print {

  .sidebar {
    display: none;
  }

  .menu {
    display: none;
  }

  .button {
    display: none;
  }

}
```

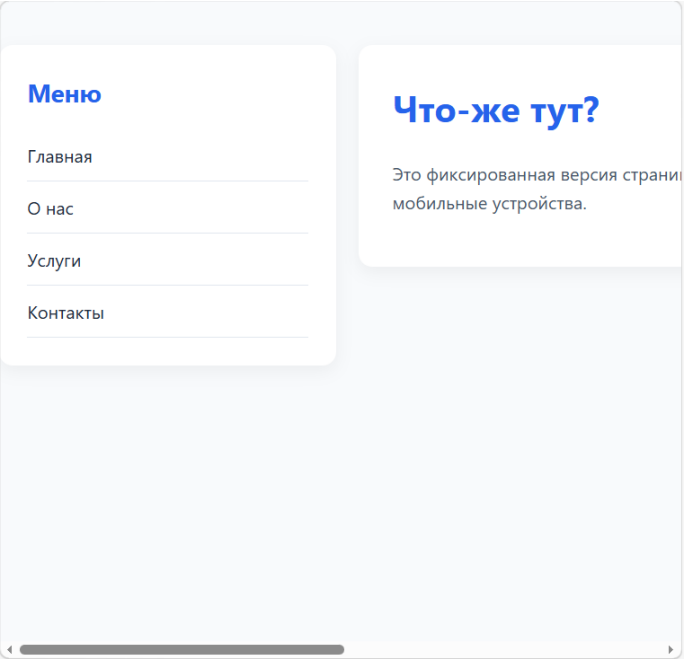
Зачем это нужно?

- экономия бумаги
- удобство чтения
- убираются ненужные элементы

Разница между фиксированной и адаптивной версткой

Фиксированная верстка:

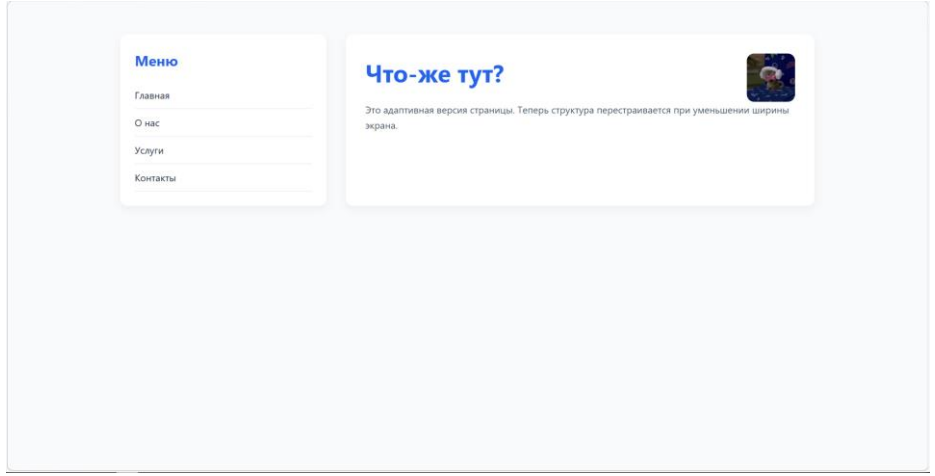


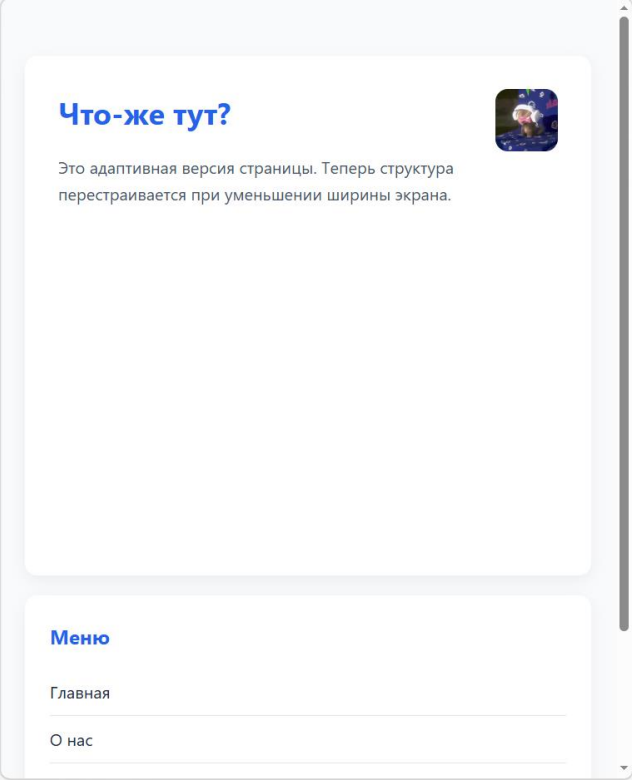


В фиксированной версии всё видно, но при уменьшении разрешения часть окна просто обрезается, так как она предназначена для ширины 1200px и более

Разница между фиксированной и адаптивной версткой

Адаптивная верстка:





В адаптивной версии при изменении ширины страница перестраивается в другой формат

Викторина

Ответы:

1) Blur

2) Нет

3) box-shadow задаёт тень блоку, а drop-shadow — по контурам элемента

4) Да

5) изменяет насыщенность цветов в изображении

6) C

7) A, C, E

8) Да

9) 1-D 2-B 3-A 4-C

Викторина

Ответы:

10) В

11) loading="lazy“

12) Да

13) Нет

14) Да

15) Да

16) Нет (она зависит от html)

17) Да

18) Да